O'REILLY®

Building Secure AI Agents with OpenClaw on AWS

Module 1: OpenClaw Architecture & AWS Deployment

Why This Course Matters

The Problem

- OpenClaw has 145K+ GitHub stars and a massive skills ecosystem
- Most tutorials stop at installation
- The opportunity is enormous; the risk surface is equally large

What You Leave With

- A running, cloud-hosted, secured OpenClaw agent
- Three custom skills you built during the course
- A production security checklist for any agent deployment on AWS

Today's schedule: Deploy → Configure → Build Skills → Add Memory → Harden Security

Meet your Instructor

Kesha Williams

Enterprise Architect & AI Consultant

Founder, Keysoft | AWS AI Hero *since 2019*

- 30+ years building production software systems
- Creator, RAISE Framework for agentic AI governance
- Early adopter of OpenClaw, currently connected to JIRA, AWS, GitHub, and OpenRouter

https://www.linkedin.com/in/keshaewilliams
@KeshaWillz
https://www.keysoft.tech

Poll: Where are you running OpenClaw today?

A Mac Mini or other local machine

B Cloud (EC2, VPS, etc.)

C Haven't set it up yet

D What is OpenClaw?

Course schedule

- **Module 1** - OpenClaw Architecture & AWS Deployment
- **Module 2** - Configuration & Messaging Integration
- **Module 3** - Building Custom Skills
- **Module 4** - Sessions, Memory & Proactive Behavior
- **Module 5** - Security Hardening on AWS

What is OpenClaw?

Three moving parts

Brain

LLM Connection

Sends prompts to an LLM provider (Bedrock, direct API, or local via Ollama). The Gateway builds the prompt, routes the response.

Hands

Tools & Skills

Executes actions: browse the web, manage calendar, run code, send emails. Skills extend what the agent can do.

Memory

Persistence

Stores context across sessions using the memory/ folder, daily diary entries, and MEMORY.md. The agent relearns itself on startup.

Gateway Architecture

Gateway (Cloud)

- Runs on Lightsail ($24/month)
- WebSocket connections + API calls
- HTTPS via Let's Encrypt (auto)
- Survives reboots (systemd)

Device Nodes (Edge)

- iMessage (needs macOS + BlueBubbles)
- Voice Wake, Talk Mode (macOS/iOS)
- Canvas, A2UI (macOS/iOS)
- Local desktop automation

Why Lightsail?

The blueprint changes the deployment story.

Before the Blueprint

- Provision EC2 instance manually
- Install Node.js, clone repo, configure
- Set up HTTPS yourself
- Create IAM roles from scratch
- Wire up Bedrock permissions manually
- Debug networking, ports, firewall

With the Lightsail Blueprint

- Select OpenClaw blueprint, pick a plan
- SSH in, pair browser, go
- HTTPS auto-provisioned (Let's Encrypt)
- One CloudShell script for Bedrock + IAM
- Bedrock is the default AI provider
- 4 GB plan, $24/month, everything included

Cost of Running OpenClaw on Amazon Lightsail

Here is a cost breakdown

Ligthsail instance

You pay a monthly fee for the instance plan you selected (e.g. $24 a month)

Tokens

Every message (sent and received) uses Amazon Bedrock's token-based pricing model.

Model Subscriptions

If you're using a third-party model through AWS Marketplace (i.e., Claude) they may be additional software fees.

Data transfer overages

Each Lightsail plan has a monthly data transfer allowance. You pay for overages on data transfer out.

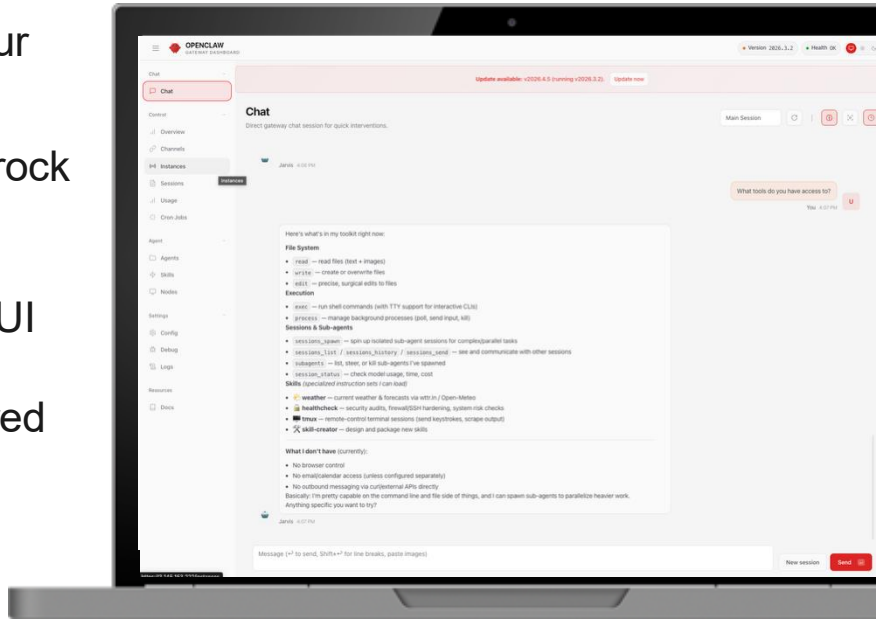
Snapshots

Manual and automatic snapshots of your Lightsail instance are billed based on storage.

Exercise: Five Steps to a Running Agent

Let's do this live. Open your AWS account and follow along.

1. Create a Lightsail instance with the OpenClaw blueprint
2. SSH in, copy the gateway token, pair your browser
3. Run the CloudShell script to enable Bedrock API access
4. Send your first message via the Control UI
5. Observe the agent's decision flow powered by Bedrock



This is the live demo + follow-along exercise.

Screen-sharing starts now.

Pulse Check

Were you able to send your first message and get a response from your agent?

Q&A

Break


O'REILLY®

O'REILLY®

Configuration & Messaging Integration

Module 2: SOUL.md, openclaw.json, and connecting Telegram

Discussion Question

Drop your answer in the chat:

What's the first thing you'd want your agent to refuse to do?

This is what SOUL.md controls. Your agent's personality, boundaries, and rules.

Configuration Anatomy

Two files control your agent's identity and behavior.

SOUL.md

Who the agent is

- Personality and identity
- Behavioral boundaries
- Communication style
- Domain focus areas
- Plain Markdown, no restart needed

openclaw.json

How the agent runs and what it can do

- tools.allow / tools.deny (permissions)
- Model selection (Bedrock, provider)
- skills.allow (skill loading)
- Memory and gateway settings
- JSON format, restart for some changes

SOUL.md

Your agent's identity document. Who it is and how it behaves.

What Goes In It

- Agent name and personality
- Behavioral boundaries (what it should refuse)
- Communication style and tone
- Domain expertise and focus areas
- Rules for interacting with users

Why It Matters

- Loaded into every prompt the Gateway builds
- First line of defense against misuse
- Plain Markdown, easy to version control
- Your agent without SOUL.md is a generic assistant

Tools

Typed functions the agent calls to interact with your system and the web.

Built-in Tools

Tool	What It Does
exec	Run shell commands, manage processes
read / write / edit	File I/O in the workspace
browser	Control a Chromium browser (navigate, click, screenshot)
web_search / web_fetch	Search the web, fetch page content
message	Send messages across all channels
cron	Manage scheduled jobs

How Tools Are Controlled

- Configured globally in openclaw.json via tools.allow and tools.deny
- Deny always wins over allow
- Tool profiles set a base allowlist (default, coding, messaging)
- Tool groups let you allow categories at once (e.g., group:fs for all file tools)
- If a tool isn't in the allow list, the agent can't use it, period

Tools Allow and Deny List

The hard permission boundary for your agent. Deny always wins.

How It Works

- Configured in the tools section of openclaw.json
- tools.allow: whitelist of tools the agent can call
- tools.deny: blacklist that always overrides allow
- Also configurable via the OpenClaw UI
- Per-agent overrides available for multi-agent setups

Why It Matters for Production

- Controls whether your agent can run shell commands (exec) on your Lightsail instance
- Controls file system access: can it read your credentials folder? Write to system paths?
- Controls web access: can it POST your data to external endpoints?
- Guards against prompt injection exploits by limiting what a hijacked agent can do
- A malicious skill can only exploit tools you've allowed

Skills

Markdown files that teach the agent when and how to use tools.

Skill Anatomy

- Each skill is a directory with a SKILL.md file
- Frontmatter: name and description (YAML)
- Body: Markdown instructions injected into the system prompt
- Optional scripts/ directory for automation
- Skills live in ~/.openclaw/workspace/skills/

How Skills Work

- The Gateway reads skill descriptions to match user intent
- When matched, the body instructions are loaded into the prompt
- Skills reference tools by name in their instructions (exec, web_fetch, read, etc.)
- Skills can only use tools that openclaw.json allows
- Installed from ClawHub or built from scratch (Module 3)

Tools vs. Skills

This distinction matters for both functionality and security.

Tools

Built-in system capabilities

- exec, read, write, browser, web_search, web_fetch
- Controlled by tools.allow/deny in openclaw.json
- Typed functions sent to the model API
- Available immediately, no installation

Security: over-permissioned tools = agent can do damage

Skills

Prompt extensions that guide the agent

- SKILL.md files injected into the system prompt
- Installed from ClawHub or built locally
- Teach the agent when and how to use tools
- Reference tools in body instructions

Security: malicious skills can misuse allowed tools

Plugins

Packages that bundle tools, skills, channels, and more together.

What Plugins Do

- Register new tools beyond the built-in set
- Add channels (messaging platforms)
- Add model providers, speech, image/video generation
- Ship skills alongside the tools they document
- Core plugins ship with OpenClaw; external plugins come from npm

How They Fit

- Tools are what the agent calls
- Skills teach the agent when and how
- Plugins package both together with extra capabilities
- Controlled via openclaw.json plugin configuration
- ClawHub has 13,000+ community-built extensions

Security: external plugins run code on your instance. Audit them the same way you audit third-party skills.

Connecting Messaging Channels

Your agent lives on Lightsail. You talk to it from your phone.

Telegram

Bot via @BotFather

Today's exercise

WhatsApp

QR code pairing

Supported

Slack

Workspace app

Supported

Discord

Bot token

Supported

Signal

Linked device

Supported

Browser

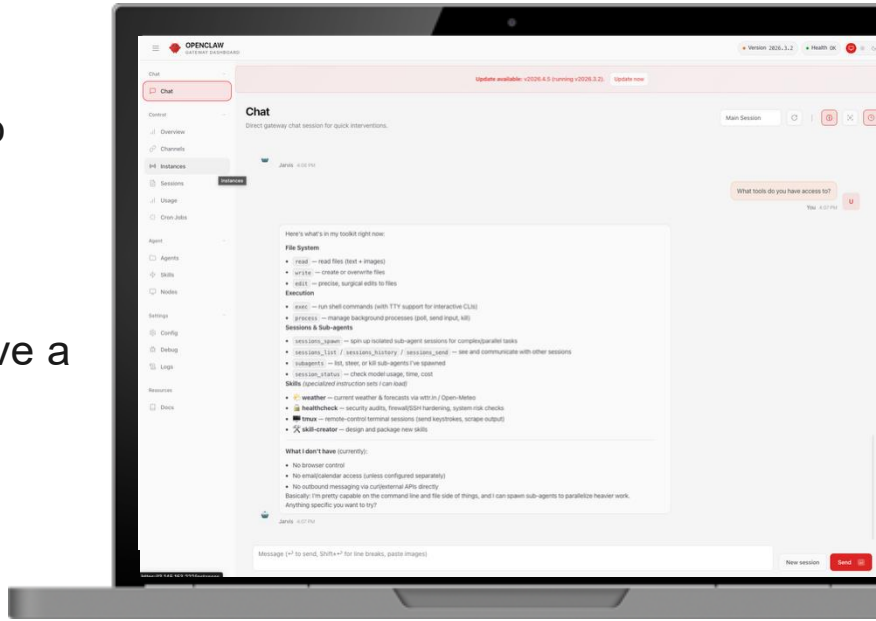
Control UI (built-in)

Already connected

Exercise: Five Steps to a Running Agent

Let's do this live. Open your AWS account and follow along.

1. Write a SOUL.md that defines your agent's personality, boundaries, and behavioral rules
2. Configure tools.allow in openclaw.json to control what the agent can do on your Lightsail instance
3. Connect your agent to Telegram and have a conversation from your phone



This is the live demo + follow-along exercise.

Screen-sharing starts now.

Pulse Check

Is your agent responding on Telegram?

Discussion Question

Drop your answer in the chat:

What surprised you about the tools vs. skills distinction?

Poll: Which messaging channel would be most useful for your work?

A Telegram

B WhatsApp

C Slack

D Discord / Teams / Other

Q&A

Break


O'REILLY®

O'REILLY®

Building Custom Skills

Module 3: Skill architecture, three skills from scratch, ClawHub publishing

Skills

YAML Markdown files (SKILL.md)

Skill Anatomy

- Each skill is a directory with a SKILL.md file
- YAML frontmatter (metadata)
- Markdown body (instructions)

Sample

```
---
name:
description:
---
# Title
…
## Steps
1. …
2. …
```

Skill Architecture

Every skill is a SKILL.md file with three layers

1

Frontmatter

YAML header: name and description. The Gateway reads this to decide if the skill is relevant to the current message.

2

Body Instructions

Markdown content loaded into the prompt when the skill is activated. Tells the LLM how to use the skill, what format to follow, and what constraints apply.

3

Supporting Scripts

Optional bash or Python scripts the skill can execute. Run in the agent's environment on Lightsail. This is where the real work happens.

Skill Precedence

When skills conflict, location wins.

HIGHEST PRIORITY

Workspace

`.openclaw/workspace/skills/`

Your custom skills. Highest priority. What we build today.

Local

`~/.openclaw/skills/`

User-level skills shared across projects.

LOWEST PRIORITY

Bundled

`Built-in with OpenClaw`

Default skills shipped with the framework. Controlled by skills.allowBundled.

There are also per-agent skills in multi-agent setups.

How Skills Use Tools

Skills reference tools in their body instructions. openclaw.json decides if those tools are available.

What the Skill Says

Skills reference tools in their body instructions. openclaw.json decides if those tools are available.

File Organizer Skill

1. Use read to list all files in the target directory
2. Use exec to create subdirectories
3. Use exec to move each file to its category folder

What openclaw.json Controls

The Gateway checks tools.allow before every tool call:

```
"tools": {
  "allow": ["read",
"web_search",
  "web_fetch"],
  "deny": ["exec"]
}
```

Skills work within the ceiling that openclaw.json sets. If you install a new skill, check what tools it references and make sure your allow list covers them.

Configuring Exec for Skills Development

The Lightsail sandbox is secure but restrictive. We need to open it up for the exercises.

Setting	Command	What it does
Exec host	`openclaw config set tools.exec.host gateway`	Commands run on the server, not inside Docker
Exec security	`openclaw config set tools.exec.security full`	Any command is allowed
Exec security	`openclaw config set tools.exec.ask off`	No approval prompt before execution

This is the dangerous configuration from the Module 5 threat model. We need it now. We'll harden it later.

If newly added skills aren’t recognized

Start a new session so OpenClaw picks up the skill

```
# From chat
/new

# Or restart the gateway
openclaw gateway restart
```

Pulse Check

Clear on the difference between frontmatter screening and body loading?

Skill 1: Project Scaffolder

Creates a full project structure with starter files. ~14 minutes.

What We're Building

- SKILL.md with frontmatter + body
- Uses exec to create directory structures
- Uses write to generate starter files
- Adapts to any project type (Python, Node.js, Go, etc.)

What to Observe

- How the agent selects this skill at runtime
- Frontmatter screening in action
- The body instructions guide file generation
- The agent adapts templates to the requested language

Discussion Question

Drop your answer in the chat:

What project type would you scaffold first for your real work?

Skill 2: File Organizer

Sorts downloads by type and date. ~12 minutes.

- Scans a target directory for files
- Categorizes by extension (images, documents, code, archives)
- Creates dated subdirectories
- Moves files into the appropriate folder
- Reports what it organized

Verify the skill loaded: openclaw skills list

Skill 3: Code Review

Analyzes a GitHub PR and flags issues. ~10 minutes.

- Fetches PR from GitHub
- Analyzes code changes for common issues
- Flags security concerns, missing tests, complexity
- Generates a structured review summary

Verify the skill loaded: openclaw skills list

Publishing to ClawHub

Share your skills with the community. 13,000+ extensions and growing.

1. Test your skill locally on your Lightsail instance
2. Write clear frontmatter with accurate tool declarations
3. Document usage examples in the body instructions
4. Publish using the OpenClaw CLI

Security reminder: third-party skills from ClawHub should be audited before installing. Module 5 covers how.

Poll: Which skill was the most useful to you?

A Project Scaffolder

B File Organizer

C Code Review

D I have a different idea

Q&A

Break


O'REILLY®

O'REILLY®

Sessions, Memory & Proactive Behavior

Module 4: How conversations persist, how memory works, and agents that act on their own

Discussion Question

Drop your answer in the chat:

Has your agent already started "remembering" things from earlier?

Session Architecture

Every conversation is a session. Sessions are the container for everything the agent knows about your interaction.

How Sessions Work

- One session per agent for DMs (default)
- Group chats get separate session keys
- Browser + Telegram = same session
- Transcripts stored as JSONL files on disk
- Gateway is the source of truth for all state

Where State Lives

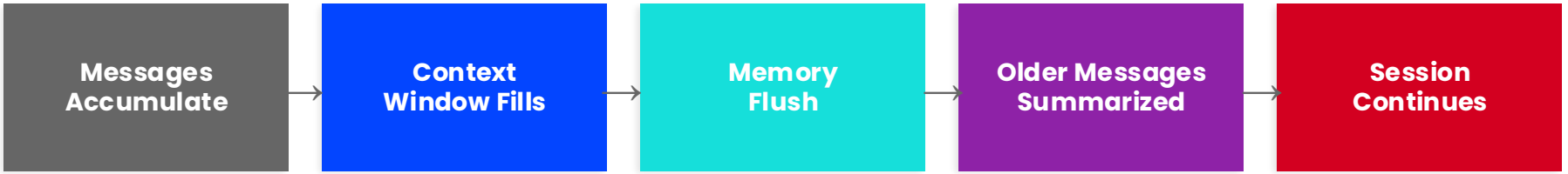
```
Session store:
~/.openclaw/agents/<id>/sessions/sessions.json

Transcripts:
~/.openclaw/agents/<id>/sessions/<sessionId>.jsonl
```

- Token counts tracked per session
- Deleting entries is safe (recreated on demand)

Compaction

What happens when your conversation exceeds the model's context window.



How Compaction Works

- Every model has a context window (max tokens). Claude Sonnet 4.6: 200K tokens.
- When the session nears that limit, compaction triggers automatically.
- Before summarizing, OpenClaw runs a silent memory flush to save important context to disk.
- Older messages are summarized into a compact entry. Recent messages stay intact.
- The summary persists in the session's JSONL transcript.
- Manual compaction: **/compact** command. Compaction vs. pruning: compaction summarizes history, pruning trims old tool results.

How Memory Works

Plain Markdown files. No hidden state. Fully transparent and editable.

1

MEMORY.md

`~/.openclaw/workspace/MEMORY.md`

Long-term memory. Durable facts, preferences, and decisions. Loaded at the start of every DM session.

2

Memory Notes

`~/.openclaw/workspace/memory/YYYY-MM-DD-topic.md`

Topic-based context and observations. Recent notes are loaded automatically.

3

Memory Search

`~/.openclaw/memory/<agentId>.sqlite`

MEMORY.md and memory/*.md indexed into chunks (~400 tokens with 80-token overlap) and stored in a per-agent SQLite database. Agent uses memory_search and memory_get tools.

Cross-Session Continuity

What This Means on Lightsail

- Session transcripts + memory files persist on the instance's 80 GB SSD
- Reboots are safe: systemd restarts the Gateway, which reloads sessions and memory
- Compaction keeps sessions bounded even for long-running agents
- Snapshots capture everything (sessions, memory, config)
- Deleting the instance deletes all state. Snapshot first.

HEARTBEAT.md

Scheduled tasks. Your agent acts without waiting for a prompt.

What It Does

- Runs tasks on a schedule using the cron tool
- Defined in `~/.openclaw/HEARTBEAT.md`
- Plain Markdown with schedule descriptions
- Results can be sent via messaging channels
- Turns your agent from reactive to proactive

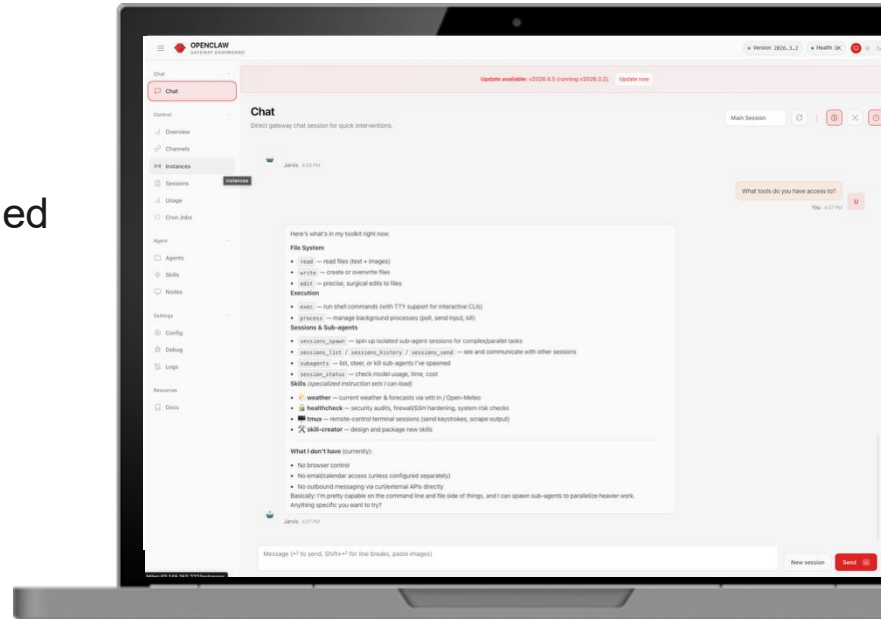
Use Cases

- Check a URL for changes every hour
- Send a daily morning briefing via Telegram
- Monitor CI/CD pipeline status
- Run a weekly security scan
- Triage overnight messages

Hands-On Exercise

Follow along. Screen-sharing starts now.

1. Explore session state and memory files on disk
2. Tell the agent something memorable and watch it write to memory
3. Set up a HEARTBEAT.md with a scheduled monitoring task
4. Restart the agent and verify it retains context across restarts



This is the live demo + follow-along exercise.

Screen-sharing starts now.

Pulse Check

Did your agent retain context after restart?

Poll: What would you schedule your agent to do proactively?

A Monitor a URL for changes

B Send daily summaries

C Check CI/CD pipeline status

D Triage incoming emails

Q&A

Break


O'REILLY®

O'REILLY®

Security Hardening on AWS

Module 5: Real audit findings, attack chains, and hardening your Lightsail instance

Poll: How many bundled skills do you think are active on your instance right now?

A 5-10

B 15-25

C 40-50

D No idea

The Three-Layer Threat Model

Based on a security audit of OpenClaw on AWS Lightsail

Layer 1: Infrastructure

What AWS gives you by default

- Unpatched OS (31 updates pending)
- Firewall open to all IPs
- IPv6 traffic bypasses IPv4 rules

You fix this with: apt upgrade, Lightsail firewall rules

Layer 2: Application

What OpenClaw exposes regardless of where it runs

- ~50 bundled skills active by default (opt-out)
- No channel access control (anyone in the group can instruct the agent)
- Memory files editable with no integrity checks
- No permission boundaries between chained agents

You fix this with: skills.allowBundled whitelist, channel allow lists, memory review

Layer 3: Configuration

Risks that only exist because of how YOU set it up

- exec host: gateway + security: full + ask: off = attacker gets your shell
- Cron jobs persist across restarts (backdoor survives reboot)
- Logs stored locally (agent can delete its own audit trail)

You fix this with: openclaw config set for exec settings, external log export, cron review

Attack Chains

A single misconfiguration isn't the problem. The chaining of weak configurations is.

Server Takeover

CRITICAL

Prompt injection → exec on gateway → no approval required → full server compromise

Persistent Manipulation

HIGH

Channel access → indirect prompt injection → memory poisoning → agent behavior permanently altered

Invisible Backdoor

HIGH

Token exposure → config access → cron job creation → backdoor survives restarts

Security Settings and Audit

Two commands every operator should run after setup.

openclaw config get

Verify your security settings

```
openclaw config get browser.enabled       → should be false
openclaw config get tools.exec.host       → should be sandbox
openclaw config get tools.exec.security   → should be allowlist
openclaw config get tools.exec.ask        → should be on-miss
openclaw config get gateway.auth     → should show redacted token
```

openclaw security audit

Automated check + fix

Checks for:

- File & folder permissions
- Access token rotation status
- Channel access controls

Fix issues automatically:

openclaw security audit --fix

Fresh blueprints flag loose file permissions. Other users on the machine can read your config.

Why AWS Matters for Agent Security

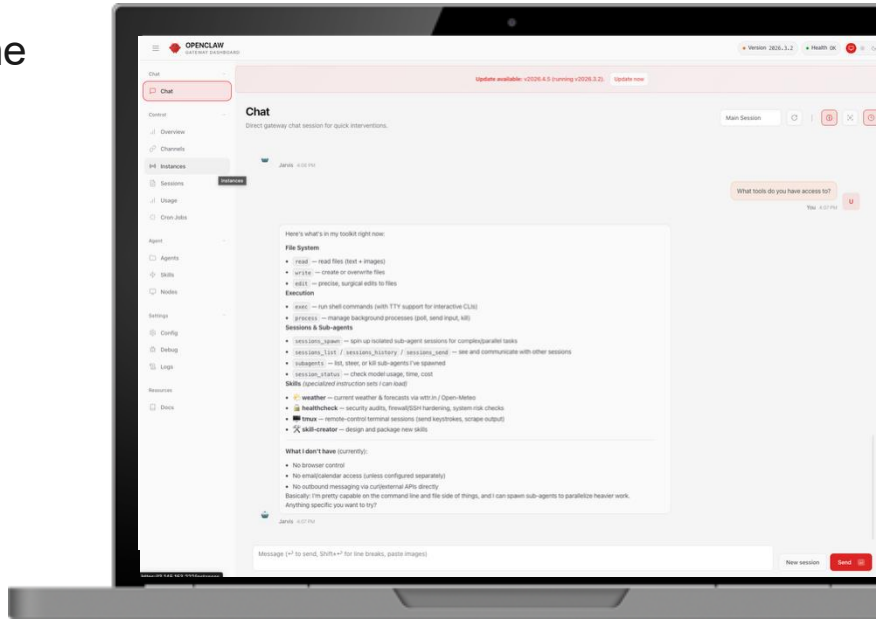
Security layers you get because you deployed on AWS, not just OpenClaw.

Bedrock Guardrails	Content filters on every API call. Block prompt attacks, redact PII (credit cards, SSNs), deny restricted topics, and apply custom regex to catch credential leaks in agent output. A defense layer between your agent and the model.
IAM Least-Privilege	Restrict which Bedrock models the agent can invoke. Remove Marketplace permissions after setup. Scope access to exactly what you need.
CloudTrail Auditing	Every Bedrock API call is logged automatically. See which model was called, when, and by whom. If something goes wrong, you have the audit trail.
Traffic Stays on AWS	Bedrock API calls never leave the AWS network. With a direct API to Anthropic or OpenAI, your prompts traverse the public internet. With Bedrock, it's all internal.

Hands-On Exercise

Follow along. Screen-sharing starts now.

1. Patch the OS, restrict the firewall, and run the security audit
2. Remove Marketplace permissions from the IAM policy
3. Create a Bedrock Guardrail with denied topics
4. Walk through the production security checklist



This is the live demo + follow-along exercise.

Screen-sharing starts now.

Pulse Check

Did your Bedrock Guardrail block the test prompt?

Discussion Question

Drop your answer in the chat:

Which attack chain worries you most for your own work?

Security Resources

Frameworks

- OWASP Top 10 for Agentic Applications 2026

genai.owasp.org

- MITRE ATLAS (OpenClaw Threat Model)

trust.openclaw.ai/threatmodel

- AWS Agentic AI Security Scoping Matrix

aws.amazon.com/ai/security/agentic-ai-scoping-matrix

Recommended Reading

- OpenClaw Security Documentation

docs.openclaw.ai/gateway/security

- OpenClaw Gateway Hardening Guide

docs.openclaw.ai/gateway

- OpenClaw Security CLI

openclaw security audit

Q&A

Break


O'REILLY®

O'REILLY®

Wrap Up

Building Secure AI Agents with OpenClaw on AWS

What You Built Today

1	**Running Agent**	OpenClaw deployed on Lightsail, connected to Bedrock, accessible via browser and Telegram
2	**Custom Identity**	SOUL.md defining personality and boundaries. tools.allow enforcing permissions in openclaw.json.
3	**Three Custom Skills**	Project scaffolder, file organizer, and code review. Built from scratch using the SKILL.md pattern.
4	**Sessions & Memory**	Session tracking, topic-based memory notes, cross-session continuity, and proactive behavior via HEARTBEAT.md.
5	**Hardened Security**	Bedrock Guardrails, least-privilege IAM, exec sandbox, security audit, and a 15-item production checklist.

Poll: What's the first thing you'll build with your agent after today?

A Automate a daily workflow

B Connect to my team's tools

C Build more custom skills

D Harden my existing setup

Recommended Next Steps

O'Reilly Courses

- **MCP Bootcamp: Building AI Agents with Model Context Protocol**

Live Online Course

- **Building Integrated AI Agents with OpenClaw**

Live Online Training

- **Building AI Agents with LangGraph**

On-Demand Course

Technical Resources

- **Course GitHub Repository**

Starter configs, skill templates, security checklists

- **OWASP Top 10 for Agentic Applications 2026**

genai.owasp.org

- **OpenClaw Docs + Security**

docs.openclaw.ai | docs.openclaw.ai/gateway/security

- **AWS Lightsail OpenClaw Quick Start**

docs.aws.amazon.com/lightsail

Takeaways

What You're Taking Home

- Exercise handouts for all 5 modules (PDF)
- 15-item production security checklist
- Three custom SKILL.md templates
- SOUL.md and HEARTBEAT.md examples
- IAM policy template for Bedrock (with ApplyGuardrail)
- GitHub repo with starter configurations
- Bedrock Guardrail setup guide

Connect

Kesha Williams

LinkedIn
X / Twitter
Website

Complete 80% or more of this course to earn your badge via Credly!

Final Q&A

O'REILLY®

Building Secure AI Agents with OpenClaw on AWS

Kesha Williams